Models and Database Management in Django

ORM, Databases, Models

Simone Ricci

Database Basics

Types, Components, Data Models

Database

A database is a **organized collection of data** that enables efficient storage, retrieval, and manipulation.

It can store different types of data as text, numbers, images, and multimedia.

Provides data to be **structured** (logical and understandable) => Data are easier to manage and manipulate.

Databases **allow sharing** data between different applications and systems (standardized store and access to data).

Controlled access to data: they can be configured to restrict access to sensitive data.

They provide mechanisms to ensure **data consistency and validity.**

They can be configured to **handle high volumes of transactions** and provide high availability and **fault tolerance** (scalability).

Database Types 1/2

Relational databases: the most widely used type of database. Based on the relational data model.

Data is organized into **Tables**. Each row in a table represents a single record. Column represents a single attribute or field of the record.

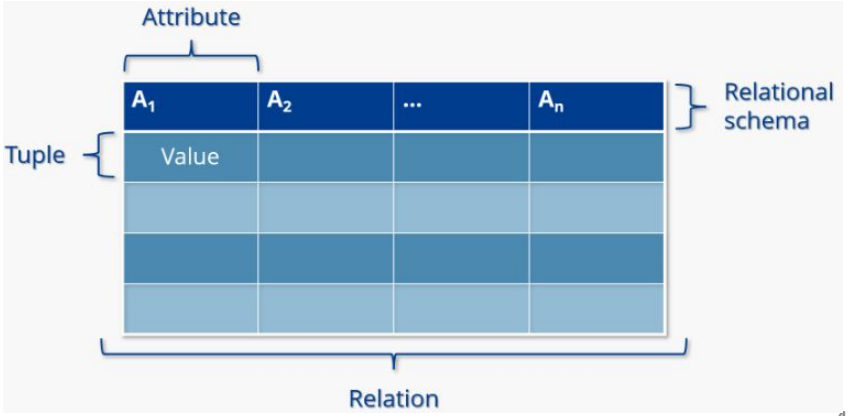
Primary key is a unique identifier for each record in the table (data integrity and to establish relationships between tables).

Foreign key can be used to link records in one table to records in another table.

Structured Query Language (**SQL**): standard language used to create, modify, and query databases.

ACID properties: Atomicity, Consistency, Isolation, and Durability.

Relational databases can scale to accommodate large amounts of data and users.



ACID

Atomicity (A): Data transactions, e.g. the entry of a new data record or the deletion of an old one, should either be **executed completely** or not at all.

Consistency (C): This property is satisfied when each data transaction moves the database from a **consistent state** to a consistent state.

Isolation (I): When multiple transactions occur simultaneously, the final state must be the same as if the transactions occurred separately.

Durability (D): The data within the database must only change as a result of a transaction and must not be changeable by external influences.

Benefit:

- It is possible for several people to work on a database without any concerns.
- Database users and developers can assume that the database is error-free and do not have to deal with troubleshooting.
- Manual debugging is no longer necessary because no errors occur.

Database Types 2/2

Non-relational databases (**NoSQL**) that use **a variety of data models**, such as key-value, document, and graph.

Designed to be flexible and scalable: **unstructured or semi-structured data** and can be easily adapted to changing data structures.

Can be **distributed** across multiple servers, which allows them to handle large amounts of data and high transaction volumes.

Use **specialized query languages** that are optimized for specific data models: MongoDB uses a query language called MongoDB Query Language (**MQL**).

Key	Document
1001	`{ "CustomerID": 99, "OrderItems": [ { "ProductID": 2010, "Quantity": 2, "Cost": 520 }, { "ProductID": 4365, "Quantity": 1, "Cost": 18 }], "OrderDate": "04/01/2017" }`
1002	`{ "CustomerID": 220, "OrderItems": [ { "ProductID": 1285, "Quantity": 1, "Cost": 120 }], "OrderDate": "05/08/2017" }`

- Relational databases for applications that require structured data with complex relationships.
- NoSQL databases for applications with large amounts of unstructured or semi-structured data.

Components of a Database

Relational database:

- **Tables**: A table is a collection of related data organized into **rows** and **columns**, and represents a specific type of object or **entity** in the database. Table expresses not only entities, but also **relations**.
- **Rows**: A row, also known as a record or tuple, represents a single instance of data in a table.
- **Columns**: A column, also known as a field or attribute, represents a single piece of data in a table.

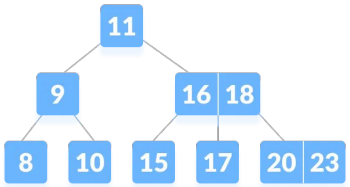
NoSQL database:

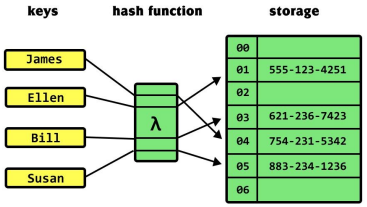
- **Documents**: A document is a **self-contained unit of data** and represents a single instance of an entity or object.
- **Collections**: A collection is a **group of related documents.**

Indexes, Queries and Transactions

They allow efficient data retrieval, manipulation, and consistency in the database.

- **Indexes**: a data structure that allows for **fast lookups of data** based on specific criteria. Speed up queries. They are typically implemented as B-trees or hash tables.
- **Queries**: is a **request for data** from a database that matches certain criteria.They are used to retrieve, filter, and sort data from one or more tables or collections.
- **Transactions**: unit of work that is executed on a database and either **succeeds or fails as a whole**.





They ensure **data consistency and integrity**.

Can involve multiple operations, such as inserting, updating, or deleting data.

Implemented using a combination of locks and logging to ensure atomicity, consistency, isolation, and durability (**ACID**) properties.

DBMS

A DBMS (**Database Management System**) is a software system that provides a way to organize, store, retrieve, and manage data in a database.

It allows access and manipulation of data in a database, while also providing security, concurrency control, and recovery mechanisms to ensure data integrity and consistency.

DBMS includes:

- **Data dictionary**: stores metadata about the database (names of tables, their columns, relationships)
- **Query language**: standardized way to retrieve and manipulate data in the database, such as **SQL** (Structured Query Language)
- **Data manipulation tools**
- **Transaction management system**

Popular DBMS: Oracle Database *(relational database)*, MySQL *(open-source, relational database)*, MongoDB *(document-oriented, NoSQL database)*, **PostgreSQL** *(open-source, relational database)*, **SQLite** (*called zero-conf, it does not require service management*).

Django's ORM (Object-Relational Mapping)

Object-Relational Mapping (ORM)

Object-Relational Mapping is an **abstraction layer** used to map object-oriented programming concepts to relational database concepts.

ORM allow define the database schema using **classes** instead of manually creating tables and fields in the database.

It allows interactions with databases using programming language objects and methods instead of writing SQL queries directly.

It provides a high-level API for performing common database operations like creating, reading, updating, and deleting (**CRUD**).

Key benefits:

- **Portability**: easy to switch between different databases without changing the application code.
- **Security**: automatically sanitizes input data and prevents SQL injection attacks.
- **Efficiency**: optimizes database queries.
- **Maintainability**: defining the database schema using Python classes makes it easier to maintain and update it over time.

Django ORM

Django ORM uses the **active record pattern** (data access logic in the domain object):

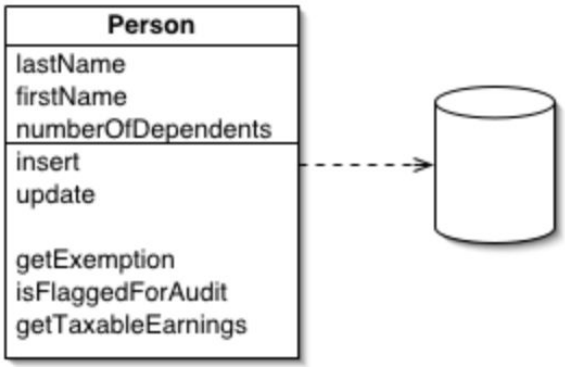
- A class maps to a single table in the database. The class is often called a **model class**.
- An object that wraps a row in a database table or view, encapsulates the database access, and adds domain logic on that data.

(Models should encapsulate every aspect of an "object," following Martin Fowler's Active Record design pattern)

Once you define a model class, you can access predefined methods to create, read, update, and delete data.

Django ORM philosophies : https://docs.djangoproject.com/en/5.2/misc/design-philosophies/#include-all-relevant-domain-logic

Django admin page lets manage also the data of the models.



Active vs Passive record pattern

Active: the domain object (model) contains both the data and the data access logic.

So, the model is “active” — it knows how to persist itself.

```
class Book(models.Model):
    title = models.CharField(max_length=100)

book = Book(title="Django 101")
book.save()  # The model instance knows how to save itself
```

Passive: the domain object only contains the data (attributes/properties).

The data access logic is separated into another layer, often called a repository or data mapper.

```
class Book:
    def __init__(self, title):
        self.title = title

class BookRepository:
    def save(self, book):
        # logic to insert/update book in database

book = Book("Django 101")
repository = BookRepository()
repository.save(book)  # Saving is handled outside the model
```

Model class

A **model** in Django is a Python class that inherits from the **django.db.models.Model** base class.

```
from django.db import models


class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

Each attribute of the model class represents a **field** in the database table.

Django table naming convention: the name of the database table comes from the name of your model class and the app that contains it.

With an app **myapp** (as created by manage.py startapp myapp), a model defined as class **Person** will have a database table named "myapp_person".

SQL

```
CREATE TABLE myapp_person (
    "id" bigint NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
    "first_name" varchar(30) NOT NULL,
    "last_name" varchar(30) NOT NULL
);
```

To override the database table name, use the **db_table** parameter in class **Meta**.

Meta class

The **Meta class** inside the model let change explicitly **metadata** options of the model.

```
class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)

    class Meta:
        db_table = 'my_custom_table_name_for_persons'
```

This will create a database table called "my_custom_table_name_for_persons" instead of the default "myapp_person".

Possible options are:

```
class Answer(models.Model):
    question = models.ForeignKey(Question, on_delete=models.CASCADE)
    # ...

    class Meta:
        order_with_respect_to = "question"
```

- **order_with_respect_to**: usually a ForeignKey
- **managed**: if False database operations **will not** be performed for this model by Django
- **app_label**: if a model is defined outside of an application in INSTALLED_APPS. Manually tells Django which app the model should be linked.
- **abstract**: if True the model will be an **abstract base class (inerithance)**. No Table created!

Model field

Not to choose field names that conflict with the models API like *clean*, *save*, or *delete*!!

Each field should be an instance of the appropriate **Field class**.

Django uses the Field class types to determine:

- **Column type**, which tells the database what kind of data to store (e.g. INTEGER, VARCHAR, TEXT).
- The **default HTML tag** to use when rendering a form field (e.g. <input type="text">, <select>).
- The **validation requirements** used in admin page and in generated forms.

Django ships with dozens of built-in field types:
(https://docs.djangoproject.com/en/5.2/ref/models/fields/#field-types)

- IntegerField(**options)
- BooleanField(**options)
- CharField(max_length=None, **options)
- DateField(auto_now=False, auto_now_add=False, **options)
- DecimalField(max_digits=None, decimal_places=None, **options)
- EmailField(max_length=254, **options)
- FileField(upload_to='', storage=None, max_length=100, **options)

Field options

A field option is a parameter that you can use to customize the behavior of a model field.

- **null**: If True empty values are stored as **NULL** in the database (Default is False). Avoid on string-based fields (They have two possible values for “no data”: NULL, and the empty string. Django convention is to *use the empty string*)
- **blank**: If set to True, the field can be left blank in Django forms.
- **default**: Sets the default value for the field if no value is provided.
- **choices**: Restricts the valid choices for the field to a **list of tuples**. The selection is enforced by model validation and the default form widget will be a select box.
- **unique**: Ensures that the value of the field is unique across all instances of the model.
- **primary_key**: If True, this field is the primary key for the model. The **primary_key=True** implies null=False and unique=True. Only one primary key is allowed on an object.

What if a CharField has both unique=True and blank=True?

What if a CharField has both unique=True and blank=True?

If **unique=True**, **blank=True** -> Only one empty string allowed, multiple blanks cause unique error

If **unique=True**, **blank=True**, **null=True** -> Multiple empty (NULL) entries allowed, uniqueness enforced on non-null values

Relations in a Database

One-to-One (1:1) -> Each record in Table A is linked to one and only one record in Table B, and vice versa.

Example: Each person has one passport; each passport belongs to one person.

When two entities share a tightly coupled relationship but are stored separately for logical or security reasons.

One-to-Many (1:N) -> One record in Table A can be related to multiple records in Table B, but each record in Table B relates to only one record in Table A.

Most common relationship type for hierarchical or parent-child data.

Many-to-Many (M:N) -> Records in Table A can be related to multiple records in Table B, and records in Table B can be related to multiple records in Table A.

When entities have multiple associations with each other.

ForeignKey (many-to-one)

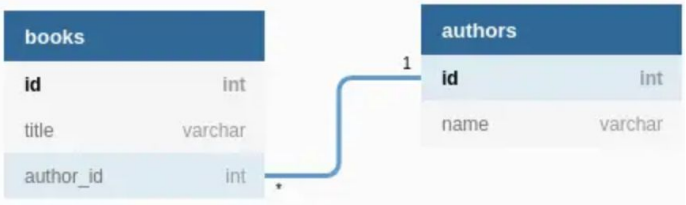
ForeignKey is a Field used to create **many-to-one relationships** within tables.

It connects tables using the primary key value from another table.

Syntax: ForeignKey(to, on_delete, **options)

- **to**: Class of connected Model
- **on_delete**: Defines the behavior of instance after the referenced instance is deleted

```
class Author(models.Model):
    name = models.CharField(max_length=512)


class Book(models.Model):
    title = models.CharField(max_length=512)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
```



ForeignKey should be placed in the Child (*many*) class, referencing the Parent (*one*) class.

Django adds **_id** suffix to the name written in the model for the database column name.

When ForeignKey and when choices?

ForeignKey vs choices in Django

ForeignKey:

- Use when you want to establish a relationship between two database tables (models).
- It creates a link to another model's instance.
- Suitable for dynamic data that can grow/change over time.
- Allows efficient querying, joins, and referential integrity.

choices:

- Use when the field can only take a limited set of predefined values.
- Suitable for static or fixed options that rarely change.
- Stored as a simple value in the database (usually a string or integer).
- Provides human-readable labels for form fields and admin.

ManyToManyField (many-to-many)

It is used to create **many-to-many relationships** within tables.

Syntax: ManyToManyField(to, **options)

- **to**: Class of connected Model

Django creates an **intermediary join table** to represent the many-to-many relationship.

This table name is generated using the name of the many-to-many field and the name of the table for the model that contains it.

```
class Publication(models.Model):
    title = models.CharField(max_length=30)

    class Meta:
        ordering = ["title"]

    def __str__(self):
        return self.title


class Article(models.Model):
    headline = models.CharField(max_length=100)
    publications = models.ManyToManyField(Publication)

    class Meta:
        ordering = ["headline"]

    def __str__(self):
        return self.headline
```

OneToOneField (one-to-one)

It is used to create **one-to-one relationships** within tables.

Conceptually, this is similar to a ForeignKey with **unique=True**, but the “reverse” side of the relation will directly return a single object.

Syntax: OneToOneField(to, on_delete, parent_link=False, **options)

- **to**: Class of connected Model
- **parent_link**: create an implicit foreign key from the child model to the parent model, instead of creating a new database column to store the relation.

The **inheritance relationship** (without abstract classes) introduces links between the child model and each of its parents (via an automatically-created OneToOneField)

```
class Place(models.Model):
    name = models.CharField(max_length=50)
    address = models.CharField(max_length=80)

    def __str__(self):
        return "%s the place" % self.name

class Restaurant(models.Model):
    place = models.OneToOneField(
        Place,
        on_delete=models.CASCADE,
        primary_key=True,
    )
    serves_hot_dogs = models.BooleanField(default=False)
    serves_pizza = models.BooleanField(default=False)

    def __str__(self):
        return "%s the restaurant" % self.place.name
```

Migrations

Migrations are Django's way of propagating changes you make to your models (adding a field, deleting a model, etc.) into your database schema.

Migration commands:

- **migrate**: used for applying and unapplying migrations.
- **makemigrations**: create new migrations file based on the changes you have made to your models.
- **sqlmigrate**: displays the SQL statements for a migration.
- **showmigrations**: lists a project's migrations and their status.

In Django, the **makemigrations** command is used to **generate new database migration files** based on the changes you have made to your models.

The **makemigrations** command **does not modify the database schema directly**.

The migration files that you can inspect and review before applying the changes to the database using the **migrate** command.

Migration files contain Python code that Django uses to apply or reverse the changes.

Migrations file

In a migration file (as a Python module) Django looks for a subclass of django.db.migrations.Migration called Migration:

```
from django.db import migrations, models

class Migration(migrations.Migration):

    dependencies = [('migrations', '0001_initial')]

    operations = [
        migrations.DeleteModel('Tribble'),
        migrations.AddField('Author', 'rating', models.IntegerField(default=0)),
    ]
```

- **dependencies**: a list of migrations this one depends on.
- **operations**: a list of Operation classes that define what this migration does.

Django uses the list of Operation to **generate the SQL** which makes the schema changes.

Filename: timestamp and a descriptive name, such as 0001_initial.py. The filename reflects the order in which the migrations should be applied to the database.

CRUD with Django ORM

CRUD (Create, Retrieve, Update, Delete) operations on models can be done using the ORM layer.

Create: to create a new object in the database, you can **instantiate a model object** and call the save() method. The create() method is used to create and save the object in a single step.

Retrieve: To retrieve one or more objects from the database, the ***objects* attribute** of the model can be used to get a **QuerySet**.

Update: after modify the fields of a model object, call the save() method to update it.

Delete: call the delete() method on the model object or the QuerySet.

CRUD examples

Model of an app:

```
class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

db.sqlite3 ▶ persons_person

Reset Filters | Records: 3

	id	first_name	last_name
1	1	Simone	Ricci
2	2	Carlo	Rossi
3	3	Marco	Giani

In the Django shell:

- **Create**:

```
>>> from persons.models import Person
>>> new_person = Person(first_name="Carlo", last_name="Rossi")
>>> new_person.save()
```

```
>>> new_person = Person.objects.create(first_name="Marco", last_name="Giani")
```

- **Retrieve**:

```
>>> persons_all = Person.objects.all()
>>> print(persons_all)
<QuerySet [<Person: Person object (1)>, <Person: Person object (2)>, <Person: Person object (3)>]>
```

```
>>> person_2 = Person.objects.get(pk=2)
>>> print(person_2.last_name)
Rossi
```

CRUD examples

Model of an app:

```
class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

db.sqlite3 ▶ persons_person

Reset Filters | Records: 3

	id	first_name	last_name
1	1	Simone	Ricci
2	2	Andrea	Innocenti
3	3	Marco	Giani

In the Django shell:

- **Update**:

```
>>> person_2.last_name="Innocenti"
>>> person_2.save()
```

```
>>> Person.objects.filter(last_name='Innocenti').update(first_name='Andrea')
```

- **Delete**:

```
>>> person_1 = Person.objects.get(pk=1)
>>> person_1.delete()
(1, {'persons.Person': 1})
```

```
>>> Person.objects.filter(last_name='Giani').delete()
(1, {'persons.Person': 1})
```

db.sqlite3 ▶ persons_person

Reset Filters | Records: 1

	id	first_name	last_name
1	2	Andrea	Innocenti

QuerySet

A QuerySet is a **list-like object** that represents a set of database records that match a certain query.

QuerySets are generated by calling methods on a **manager attribute** (**objects**) of a model, which in turn generate a SQL query to retrieve the appropriate records from the database.

Support **chaining** of methods to build complex queries, allowing you to filter, order, and limit the results.

QuerySets are **lazy**, which means that they do not actually retrieve data from the database until you need to access the data.

It's possible to define a custom manager for a model by creating a subclass of the **models.Manager** class and adding it as an attribute to a model class.

```
class MyModelManager(models.Manager):
    def get_by_name(self, name):
        return self.get(name=name)

class MyModel(models.Model):
    name = models.CharField(max_length=255)
    # other fields ...

    objects = MyModelManager()
```

Custom Manager: to add extra Manager methods, and/or to modify the initial QuerySet the Manager returns.

QuerySet

A QuerySet is **iterable**, and it executes its database query the first time you iterate over it:

```
for e in Entry.objects.all():
    print(e.headline)
```

QuerySet can be sliced, using Python's array-slicing syntax.

Django gives QuerySet refinement methods that modify either the types of results returned by the QuerySet or the way its SQL query is executed.

Some of them:

- **filter(*args, **kwargs)**: returns a QuerySet containing objects **that match** the given parameters. Multiple parameters are joined with AND in the SQL statement.
- **exclude(*args, **kwargs)**: returns a QuerySet containing objects **that do not match** the given parameters.
- **order_by(*fields)**: results returned by a QuerySet are ordered by the ordering tuple given by the ordering option in Meta or by the fields passed as argument.
- **all()**: return a QuerySet that represents all the objects in a model's table. It is useful for chaining methods and building complex queries.

QuerySet example

Here are some examples of function that return a QuerySet:

```
class MyModel(models.Model):
    name = models.CharField(max_length=255)
    # other fields ...
```

A basic model class definition.

```
# Retrieve all objects of a model
queryset = MyModel.objects.all()

# Retrieve objects that match a filter condition
queryset = MyModel.objects.filter(name='John')

# Retrieve objects that match multiple filter conditions
queryset = MyModel.objects.filter(name='John', age=30)

# Order the objects by a field
queryset = MyModel.objects.order_by('name')
```

```
# Reverse the order of the objects
queryset = MyModel.objects.order_by('-name')

# Limit the number of results
queryset = MyModel.objects.all()[:5]

# Get a single object or raise an exception if not found
obj = MyModel.objects.get(pk=1)

# Get a single object or return None if not found
obj = MyModel.objects.filter(pk=1).first()
```

Manage Models with Admin Page

Django offers robust **admin interface** that provides a visual way to interact with model data.

Django was originally built as a **newspaper CMS** (Content Management System). The idea was that journalists could write and edit their stories in the admin without needing to touch "code."

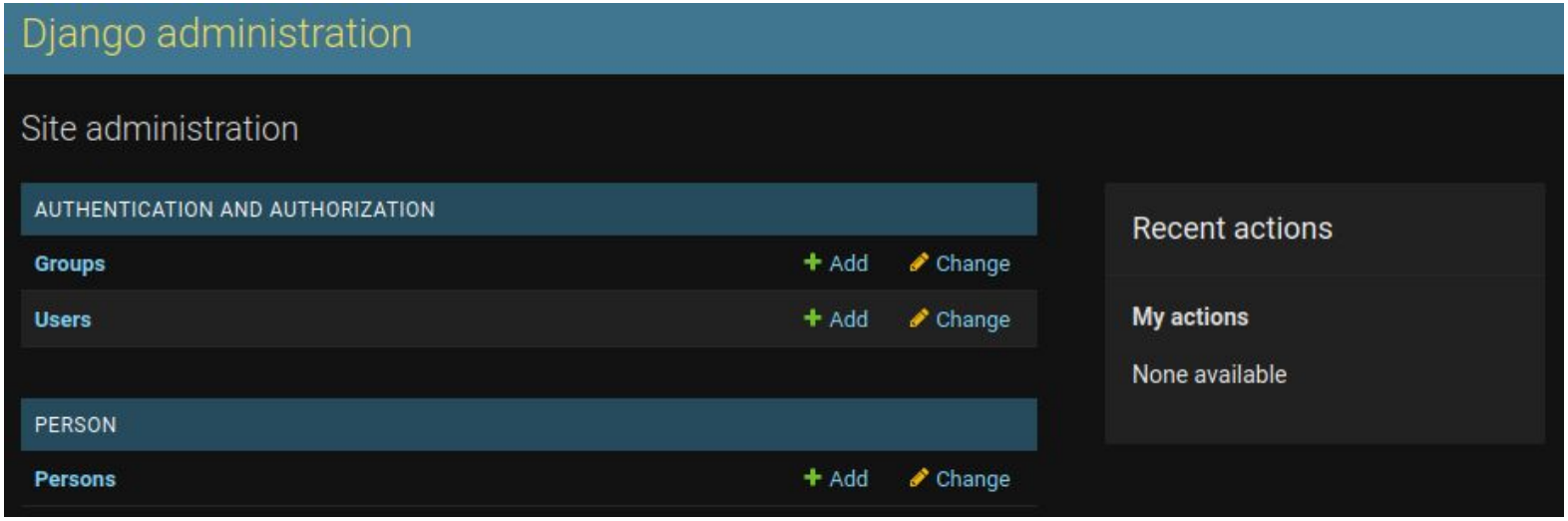
To use the Django admin, we first need to create a superuser who can log in. With python manage.py createsuperuser a superuser can be created.

Just as we must explicitly add new apps to the INSTALLED_APPS config, so, too, must we update an app's **admin.py** file for it to appear in the admin.

In **app_name/admin.py**, we need to add the following code so that the Person model is displayed.

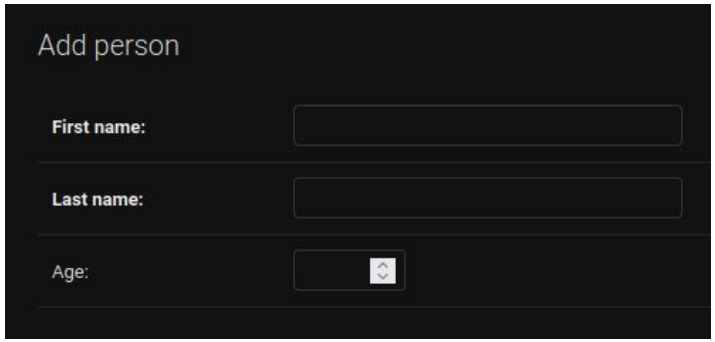
```
from django.contrib import admin

# Register your models here.

from .models import Person

admin.site.register(Person)
```

Manage Models with Admin Page



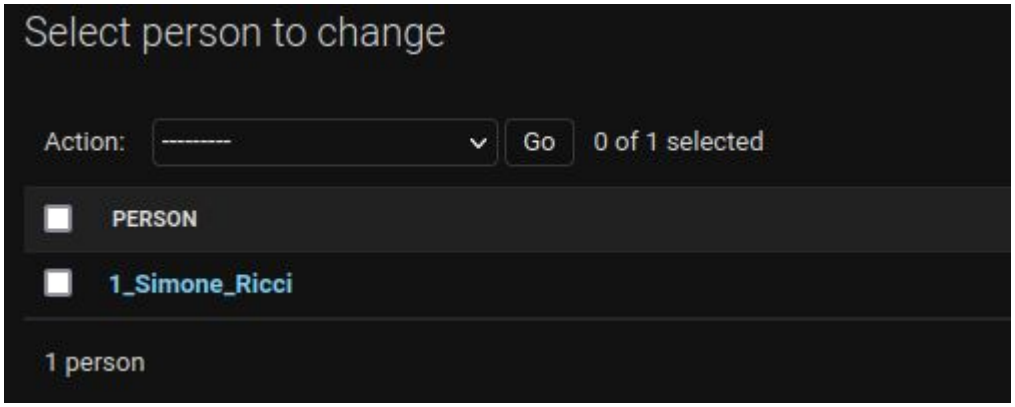
To access to the admin page we need to visit http://127.0.0.1:8000/admin/

We can view our model data and do CRUD operation on them by the admin interface



Manage Models with Admin Page

If i have some data already saved i can access them.



The “name” of each data displayed can be customized defining the __str__(self) method.

```
from django.db import models

# Create your models here.

class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
    age = models.IntegerField(blank=True, null=True)

    def __str__(self):
        return str(self.pk) + "_" + self.first_name + "_" + self.last_name
```

Populate database with fixtures

In Django, fixtures are a way to populate a database with initial data for **testing or development** purposes.

A fixture is a file that contains serialized data in one of several formats, such as JSON, YAML, or XML, that can be loaded into a Django database using the python manage.py loaddata command.

How to create a fixture for a Django model:

- Create a file named myfixture.json or myfixture.yaml in a folder named fixtures inside the app directory.
- In the fixture file, **define the initial data as a list of objects**, where each object represents a record in the database. The keys of each object **should** correspond to the field names of the model.

```
[
    {
        "model": "myapp.book",
        "pk": 1,
        "fields": {
            "title": "The Great Gatsby",
            "author": "F. Scott Fitzgerald",
            "published_date": "1925-04-10"
        }
    },
```

Deployment on Railway

Deployment

Deployment is the process of taking the software that was developed and making it available to users.

During deployment, the software is installed on servers or other infrastructure, configured to work in a **production environment.**

The deployment environment is optimized for stability, scalability, and security.

Deployment environments typically have more robust infrastructure, such as load balancers and multiple servers, to handle high traffic and ensure availability.

Deployment environment:

- **Railway**
- Heroku
- AWS Elastic Beanstalk
- Google Cloud Run
- DigitalOcean App Platform
- Netlify

Railway

Railway is a cloud-based platform that provides an easy and scalable way to deploy and manage their web applications.

It is a **platform as a service (PaaS)** that takes care of infrastructure, deployment, and scaling, allowing developers to focus on building their applications.

It integrates with popular tools like GitHub, GitLab, and Bitbucket, making it easy to deploy applications directly from a code repository.

Railway also offers a free tier…

The starter plan environment has 512 MB of RAM and 1 GB of storage memory.

Many of the Django project settings should be different for production, either for security or performance reasons.

Railway

Web applications run in their own isolated and independent virtualized **container**.

Railway needs to be able to set up the appropriate environment and dependencies, and also understand how it is launched.

For Django apps we provide this information in a text files:

- **runtime.txt**: states the programming language and version to use.
- **requirements.txt**: lists the Python dependencies needed for your site, including Django.
- **Procfile**: A list of processes to be executed to start the web application.
- **wsgi.py**: WSGI configuration to call our Django application in the Railway environment.

Once the application is running it can configure itself using information provided in **environment variables**.

Developers interact with Railway through the Railway site, and using a special Command Line Interface (CLI) tool.

Railway files example

runtime.txt

runtime.txt

You, 1 second ago | 1 author

```
python-3.12.3
```

requirements.txt

Could be generated with:
pip freeze > requirements.txt

requirements.txt

```
asgiref==3.8.1
dj-database-url==2.1.0
Django==4.2.11
gunicorn==22.0.0
packaging==24.0
psycopg2-binary==2.9.9
setuptools==68.2.2
sqlparse==0.5.0
typing_extensions==4.11.0
wheel==0.41.2
whitenoise==6.6.0
```

Procfile:

```
web: python manage.py makemigrations && python manage.py migrate && python manage.py collectstatic --noinput && gunicorn django_project.wsgi --log-file -
```

wsgi.py

```
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'django_project.settings')

application = get_wsgi_application()
```

Gunicorn

Gunicorn 'Green Unicorn' is a **Python WSGI HTTP Server for UNIX**. The Gunicorn server is broadly compatible with various web frameworks, simply implemented, light on server resources, and fairly speedy.

pip install gunicorn

Database URL

Use a database that runs in its own process somewhere on the Internet.

It is accessed by the Django library application using an **address** passed as an environment variable.

The database connection information is stored in the environment variable **DATABASE_URL**.

dj-database-url package parse the **DATABASE_URL** environment variable and automatically convert it to Django's desired configuration format.

psycopg2 package allows Django to interact with **Postgres** databases.

pip install dj-database-url psycopg2

Critical settings check

It is common to have a separate **settings.py** file for production, and/or to conditionally import sensitive settings from a separate file or an **environment variable.**

A full checklist of settings you might want to change is provided in Deployment checklist.

It is possible to visualize list a number of these with python manage.py check --deploy

- **DEBUG**: it should be set as False in production (DEBUG = False). This stops the sensitive/confidential debug trace and variable information from being displayed.

```
# DEBUG = True
DEBUG = os.environ.get('DJANGO_DEBUG', '') != 'False'
```

- **SECRET_KEY**: it is a large random value used for CSRF protection, etc. It is important that the key used in production **is not in source code or accessible outside the production server**. Django suggests that this might best be loaded from an environment variable or read from a server-only file.

```
# SECRET_KEY = "cg#p$g+j9tax!#a3cup@1$8obt2_+&k3q+pmu)5%asj6yjpkag"
import os
SECRET_KEY = os.environ.get('DJANGO_SECRET_KEY',
'cg#p$g+j9tax!#a3cup@1$8obt2_+&k3q+pmu)5%asj6yjpkag')
```

Command to generate a key:

```
python -c "import secrets; print(secrets.token_urlsafe())"
```

Database and Host settings

We need to update the setting.py file for handle the **DATABASE_URL**.

```
# Update database configuration from $DATABASE_URL.
import dj_database_url
db_from_env = dj_database_url.config(conn_max_age=500)
DATABASES['default'].update(db_from_env)
```

Django will now use the database configuration in **DATABASE_URL** if the environment variable is set; otherwise it uses the default SQLite database.

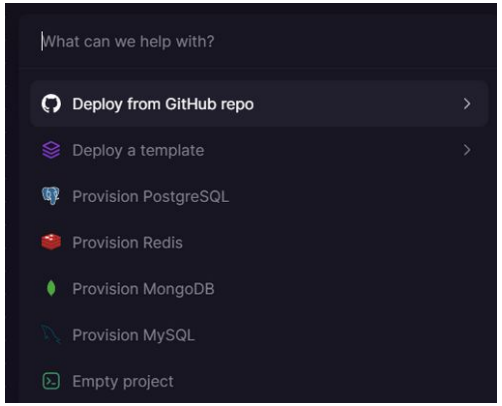
If we run the code on Railway, Django gives us a **security error** that is raised because our source code is not running on an "allowed host".

```
ALLOWED_HOSTS = ["djangorailway-production.up.railway.app", "127.0.0.1", "localhost"]

CSRF_TRUSTED_ORIGINS = ['https://djangorailway-production.up.railway.app']
```

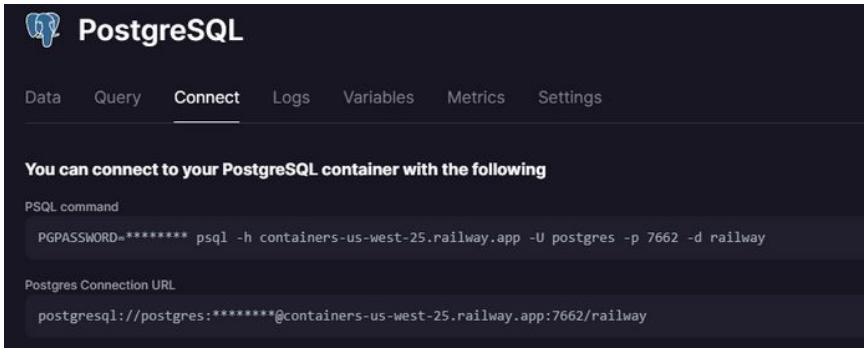
Github and PostgreSQL

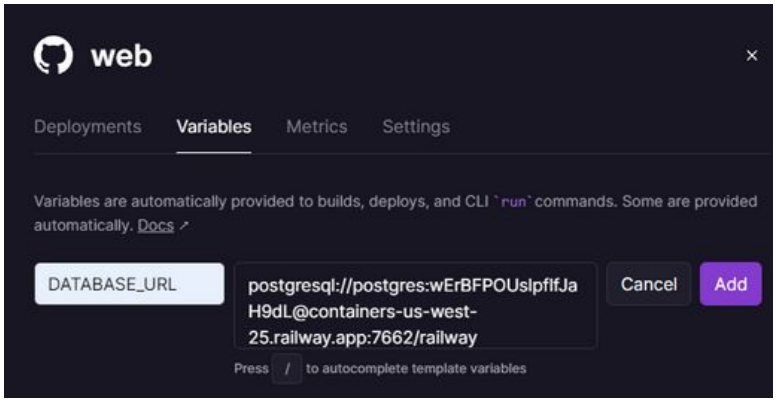
Railway will display a list of options for the new project: deploy a project from a GitHub repository, and a number of databases.

Connect your Github account with Railways to deploy.



We can create a PostgreSQL database and connect it to our deployed github project with **DATABASE_URL** variable.





Railway CLI

If we want to control our deployed application we can use the Railway CLI.

For example:

- In order to create a superuser, we need to call the Django createsuperuser command against the production database.

CLI command:

- railway login: log to your Railway account
- railway link: select/enter a particular project
- railway run: run code locally
- railway up: deploy the linked project directory
- railway logs: show the tail of logs (a full log is available on the site for each project)

To solve our example task we can use:
railway run python manage.py createsuperuser

Live Deployment